AUTONOMICS

함께 탄 만큼, 차가 실제로 쓴 만큼.

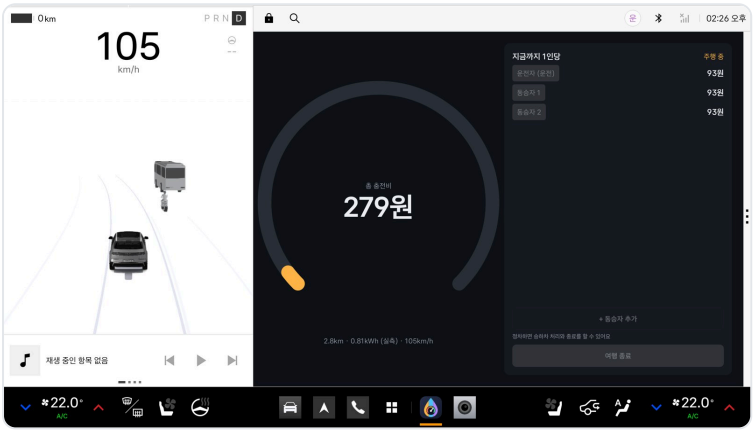
Pleos 차량 데이터로 실제 소모한 연료·전기 값을
탑승 구간별로 나누는 차량용 정산 앱

01 Executive Summary 한눈에 보기

받치는 심사 항목
전체 · 첫인상

한 문장 정의

함께 차를 탄 사람들이, 차가 실제로 쓴 연료·전기 값을 각자 탄 구간만큼 나눠 내도록 — 운전자가 계산하지 않아도 내리는 순간 금액이 확정되는 서비스.



주행 중 — 차는 105km/h로 달리고, 앱은 배터리가 실제로 줄어든 만큼 금액을 올린다. 버튼은 잠겨 있다.



도착 — 사람별 금액, 구간별 내역, 지나온 길, 동승자용 QR.

Problem 해결할 불편	동승 이동의 기름값·충전비는 **나눌 총액을 아무도 모른다.** 그래서 어림으로 N빵을 하고, 먼저 내린 사람도 같은 돈을 내고, 운전자는 말을 못 꺼낸다.	02–03
Why In-Car 차량만의 기회	그 총액을 아는 것은 **차뿐이다.** 휴대폰은 이 차가 연료·전기를 얼마나 썼는지 알 수 없다.	04
Solution 새로운 차량 경험	차가 잰 실제 소모량 × 오늘 단가를 **탑승 구간마다, 그때 타고 있던 사람끼리** 나눈다. 내리는 순간 금액이 확정되고, 동승자는 설치 없이 QR만 찍는다.	05–07
Feasibility 실제 구현	Pleos SDK 3종을 실제로 연동한 앱이 **에뮬레이터에서 처음부터 끝까지 동작**한다. 정산 규칙은 단위 테스트 46개로 검증했다.	08–09
Growth 사업성과 확장	동승자에게 건네는 **QR 정산서가 다음 사용자를 데려오고**, 혼자 탈 때의 솔로 모드가 다음 운전에 다시 열게 한다.	10–12

3종
실제 연동한 Pleos SDK
Vehicle · NaviHelper · SpeechToText

46개
정산·구간 규칙 단위 테스트
전부 통과 (2026-11-10 실행)

0회
주행 중 필요한 조작
승하차 처리는 정차 중에만

0개
동승자가 설치할 앱
QR → 폰 브라우저 정산서

플랫폼	Pleos Connect (Android Automotive OS, API 34) · Kotlin + Jetpack Compose
사용 Pleos SDK	Vehicle 2.3.2 · NaviHelper 3.0.5 · SpeechToText 3.3.1 (+ AAOS CarPropertyManager)
개발 상태	핵심 기능 구현 · Pleos Connect 에뮬레이터(v2.1, 아이오닉 5) 검증 완료
함께 내는 것	시연 영상 · GitHub 저장소(소스·APK·실행 방법) 링크 기입

이 문서의 표기 사실 직접 측정했거나 출처가 있는 수치 · 구현 코드와 시연으로 확인되는 기능 · 가정 검증 전 예상치와 목표. 화면은 모두 에뮬레이터에서 실제 Pleos SDK 값으로 찍었다.

기름값·충전비를 "얼마씩" 받아야 하는지, 차에 탄 누구도 정확히 모른다.

금액을 모르니 대충 나누거나, 먼저 내린 사람도 똑같이 내거나, 운전자가 말을 못 꺼내고 손해를 본다.

고객 인구통계가 아니라 "차에 누가, 어떻게 타는가"로 정의

이용 상황	모습	정산의 어려움
A. 여행·귀성 장거리	친구 2~4명이 한 차로 수십~수백 km 이동. 운전자 차, 운전자 카드로 주유·충전	금액이 커서 대충 나누기 껄끄럽다
B. 정기 카풀	동료·이웃이 출퇴근을 같이. 매번 타는 사람이 조금씩 다르다	소액이 반복돼 계산·청구가 귀찮다
C. 경유·중간 하차	가는 길에 한 명을 먼저 내려 주거나, 중간에 한 명을 더 태운다	"누가 어디까지 탔는지"를 계산할 방법이 없다

문제가 생기는 순간

출발 전
얼마 받을지 모른다
"나중에 정산하자"

→

중간 하차
내리는 사람 몫을
그 자리에서 못 정한다

→

도착 후
주유·충전 시점과 이동 구간이
달라 영수증이 안 맞는다

→

며칠 뒤
운전자가 계산·청구
또는 그냥 포기

지금의 해결 방법과 한계

현재 방식	한계
"기름값 3만 원씩" 어림 N빵	실제 소모와 무관하다. 먼저 내린 사람도 같은 금액 → 불공정하다고 느끼지만 말하기 어렵다
주유 영수증 나누기	주유는 이동 전후에 따로 하고, 남은 연료는 반영되지 않는다. 전기차는 집 충전·급속 충전이 섞여 이동과 맞는 영수증이 없다
공인 연비 × 거리 계산	시내·고속·정체·공조·기온에 따라 실제 효율은 공인 값과 다르다. 계산도 운전자 몫이다
더치페이·송금 앱	나누기와 송금은 해 주지만 **나눌 총액**은 사람이 알아내 입력해야 한다

같은 차, 세 사람의 불편

운전자
"얼마 달라고 하지?" 근거가 없어 말을 꺼내기 어렵다. 계산도 청구도 내 몫이라, 결국 덜 받거나 안 받는다.

끝까지 탄 동승자
"얼마를 줘야 하지?" 적게 주면 실례일까 눈치를 본다. 정확한 금액만 알면 바로 보낼 텐데.

먼저 내린 동승자
"절반만 탔는데…" 똑같이 내자니 아깝고 따지자니 미안하다. 내릴 때 금액을 모르니 나중으로 미뤄진다.

근본 원인

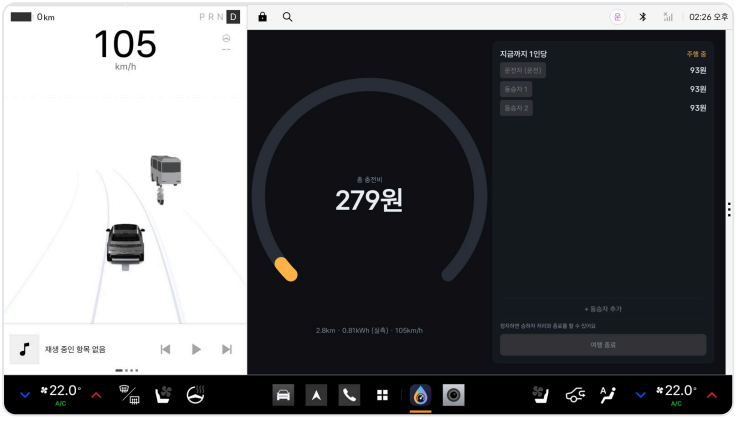
네 가지 방법이 막히는 지점은 같다. **이번 이동에 실제로 든 연료·전기값, 곧 "나눌 총액"을 아는 사람이 없다.** 영수증은 이동과 시점이 다르고, 공인 연비는 실제와 다르며, 누가 어디까지 탔는지는 기억에 의존한다. 이 값을 처음부터 끝까지 알고 있는 것은 차뿐이다. → 04

예시 장면

금요일 퇴근길, 운전자가 동료 둘을 태운다. 한 명은 중간 지하철역에서 내린다. "기름값 얼마 줄까?" "됐어, 다음에 밥 사." 다음 주에도 같은 대화가 반복되고, 결국 누구도 정확한 금액을 모른 채 운전자가 부담한다.

(조사 결과 반영 예정)

휴대폰은 이 차가 연료·전기를 얼마나 썼는지 모른다. 이 정산은 차량 데이터 없이는 계산 자체가 성립하지 않는다.



실제 Pleos Connect 화면 (에뮬레이터 · 아이오닉 5). 왼쪽은 계기판, 오른쪽이 AUTONOMICS.

이 한 장면에 차량이어야 하는 이유가 다 있다

1. **차가 잰다** — 달리는 동안 배터리 에너지가 실제로 줄어든 양(0.81kWh)을 읽어 금액(279원)을 올린다. 휴대폰이 알 수 없는 값이다.
2. **차가 안다** — 지금 105km/h로 주행 중임을 차속·기어로 안다. 그래서 이름 버튼·동승자 추가·여행 종료가 모두 잠겨 있다.
3. **차가 이어 간다** — 앱을 열어 두지 않아도, 휴게소에서 시동을 껐다 켜도 기록은 계속된다.

같은 상황, 다른 경험 지금의 방식과 차 안에서의 방식

비교 기준	지금의 방식 (휴대폰 · 사람의 계산)	AUTONOMICS (차 안에서)	이를 가능하게 하는 차량 데이터
문제를 푸는 시점	도착한 뒤나 며칠 뒤, 운전자가 따로 계산한다	달리는 동안 차가 기록하고, **내리는 순간 그 자리에서** 금액이 확정된다	정차 판단 (차속 · 기어)
사용자 조작	총액 알아내기 → 앱에 입력 → 구간·인원 계산 → 청구	출발 전 이름을 **말하고**, 내릴 때 이름을 **한 번 누른다**	SpeechToText (온디바이스)
쓰는 데이터	주유 영수증, 공인 연비 × 지도 거리 (추정)	배터리 에너지·연료 잔량의 **실제 변화**와 주행거리계	Vehicle SDK EvBattery · Fuel · Odometer
경험의 차이	근거가 없어 청구를 망설이고, 먼저 내린 사람도 같은 금액을 낸다	근거가 화면에 보이고, **탄 구간만큼만** 낸다	구간별 실측 에너지
차량만의 이점	앱을 켜 둬야 하고, 운전자가 폰을 들어야 한다	시동을 껐다 켜도 이어지고, **주행 중에는 조작이 잠긴다**	부팅 신호 · 주행거리계 보정

차량 특화성을 가르는 세 가지 질문

Q1. 차량 데이터가 없으면?

핵심 기능이 사라진다. 금액의 근거인 실제 소모량이 없어져 공인 효율 추정으로 떨어진다. 그래서 앱은 차가 값을 주지 못한 구간만 추정하고, 화면에 **실측/추정**을 구분해 보여 준다.

Q2. 주행·정차에 따라 달라지나?

그렇다. 주행 중에는 금액만 실시간으로 보이고 버튼이 잠긴다. 정차하면 하차 정산·동승자 추가·종료가 열린다. 같은 화면이 차의 상태에 따라 바뀐다. → 07

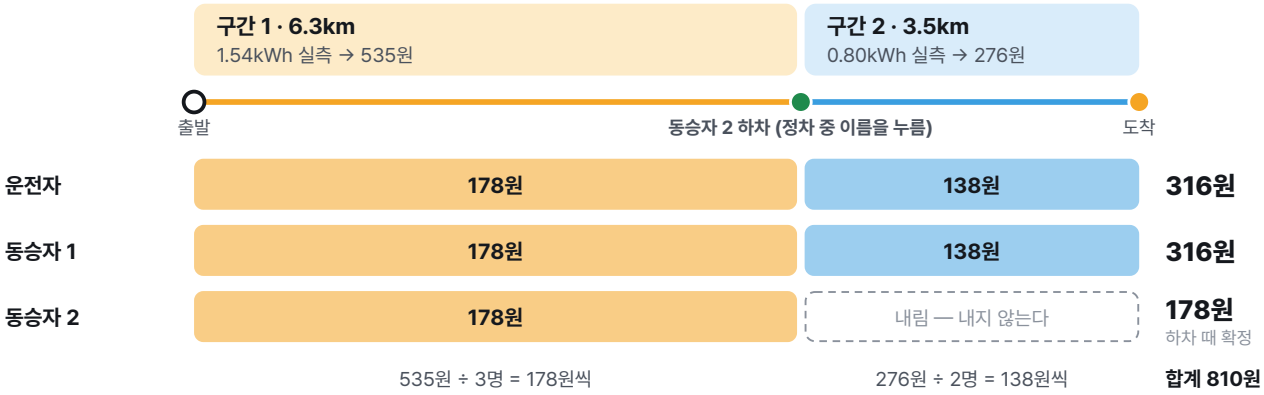
Q3. 주의 분산은?

운전 중 입력 0회. 이름은 출발 전 음성으로, 승하차는 정차 중에만, QR은 동승자가 찍는다. 기록은 포그라운드 서비스가 하므로 운전자가 화면을 볼 필요가 없다.

차량 API를 "쓴다"가 아니라, 차량 데이터가 **없으면 성립하지 않는** 서비스다. 이 앱을 스마트폰으로 옮기면 남는 것은 사람이 입력한 총액을 나누는 계산기뿐이다.

정산의 단위를 "총액"에서 "탑승 구간"으로 바꾼다. 차가 재고, 탄 만큼 나누고, 내리는 순간 끝낸다.

한 사람의 몫 = Σ (탄 구간의 실제 비용 ÷ 그 구간에 타고 있던 인원) · 구간의 실제 비용 = 차가 잰 소모량(kWh·L) × 오늘 단가



사실 에뮬레이터 시연 실측 (9.8km · 2.34kWh · 단가 347.2원/kWh, 환경부 공용 급속 고시). 에뮬레이터의 배터리 감소는 주행 속도에 맞춘 물리 모델로 움직였고, 앱은 그 값을 실차와 같은 Pleos SDK로 읽었다. 사람별 금액은 각각 1원 단위로 정하고 총액은 그 합이다(구간 표시 금액의 합과 1원 차이는 반올림).

> **같은 이동, 두 가지 계산**
>
> 어림 N빵이면 810원 ÷ 3 = **세 사람 모두 270원**. 구간 정산이면 먼저 내린 동승자 2는 **178원**(N빵보다 34% 적게), 끝까지 탄 두 사람은 **316원**. 금액이 작아 보이는 것은 시연 거리가 9.8km이기 때문이고, 비율은 거리가 늘어도 같다.

핵심 가치 세 가지 주장 → 근거 → 고객 가치

주장	근거 (차량 데이터 · 앱의 처리)	고객 가치
① 차가 잰다 추정이 아니라 실측	Vehicle SDK의 배터리 에너지(Wh)·연료 잔량 변화를 구간마다 기록한다. 값을 받지 못한 구간만 공인 효율로 추정하고 화면에 "추정"으로 표시한다. 구현	"얼마 받아야 하지?"라는 계산과 망설임이 사라진다
② 탄 만큼 나눈다 총액이 아니라 구간	타고 내릴 때마다 구간을 끊고, 각 구간의 비용은 그때 차에 있던 사람끼리만 나눈다. 중간에 탄 사람은 그때부터만 낸다. 규칙은 단위 테스트로 검증했다. 구현	먼저 내린 사람도, 나중에 탄 사람도 금액을 납득한다
③ 내릴 때 끝낸다 며칠 뒤가 아니라 그 자리	정차 중 이름을 누르면 그 사람의 금액이 확정되고 QR이 뜬다. 동승자는 폰 카메라로 찍어 브라우저에서 정산서를 열고 금액을 복사한다. 설치가 필요 없다. 구현	운전자가 나중에 계산해서 청구하던 일이 없어진다

거리가 길어지면 주말 왕복 여행으로 옮겨 본 예시 가정

운전자 포함 4명 · 그중 1명은 가는 길만	어림 N빵	구간 정산	차이
가는 길만 함께 탄 동승자	약 17,000원	**약 8,500원**	N빵이면 탄 거리의 두 배를 낸다
왕복한 세 사람 (각자)	약 17,000원	**약 19,800원**	N빵이면 2,800원씩 덜 낸다

가정 — 휘발유차, 편도 220km 왕복, 실제 연비 12km/L, 휘발유 1,855원/L(2026-11-10 오피넷 전국 평균), 총 약 68,000원. 실제 금액은 차가 잰 소모량으로 정해진다.

함께 제공하는 것 — 전기·휘발유·경유·하이브리드 지원, "운전자도 나눠 내기" 켜고 끄기, 혼자 탈 때 같은 실측으로 구간별 운전 효율을 보여 주는 솔로 모드(→ 07).

운전자는 출발 전에 이름을 말하고, 내릴 때 한 번 누른다. 그 사이의 기록과 계산은 차가 한다.

대표 시나리오 — 운전자가 친구 둘을 태우고 출발해, 한 명을 가는 길에 내려 주고 나머지와 목적지까지 간다.

상황 발생
"기름값 얼마씩 내면 돼?" — 아무도 모른다

→

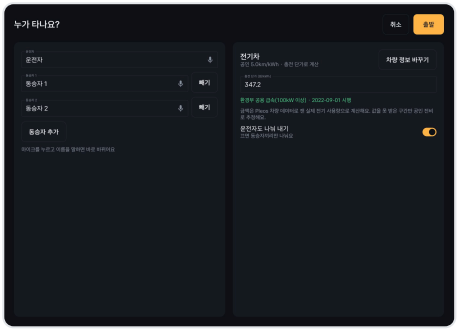
서비스 작동
차가 실제 소모량을 재고, 승하차마다 구간을 나눈다

→

문제 해결
내리는 순간 금액이 확정되고, QR로 건넨다

1 출발 전 — "누가 타나요?" 정차 중

- **기존 불편** 얼마 받을지 몰라 "나중에 정산하자"로 미룬다
- **사용자** 정산 모드를 누르고, 마이크를 눌러 동승자 이름을 말한다
- **차 · 앱** 차종을 차량 데이터로 판별해 두고, 오늘 유가·충전 단가를 출처·날짜와 함께 채운다 SpeechToText · Vehicle · 가격 API
- **얻는 것** **타이핑도 단가 검색도 없이 출발한다**



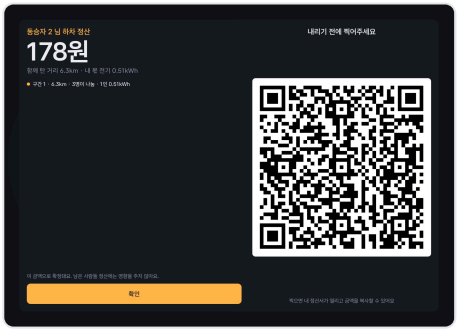
2 주행 중 — 보기만 한다 주행 중

- **기존 불편** 주유·충전 시점과 이동 구간이 달라 영수증이 맞지 않는다
- **사용자** 그냥 운전한다 (조작 0회)
- **차 · 앱** 1초마다 배터리 에너지·주행거리를 읽어 총액과 1인당 금액을 올린다. 버튼은 잠긴다 Vehicle EvBattery · Fuel · Odometer · Gear, AAOS 차속
- **얻는 것** **신경 쓰지 않아도 실제 소모량이 쌓인다**



3 중간 하차 — 내리는 순간 확정 정차 중

- **기존 불편** 먼저 내리는 사람 몫을 그 자리에서 정할 방법이 없다
- **사용자** 정차 후 내리는 사람 이름을 누른다. 내리는 사람이 QR을 폰으로 찍는다
- **차 · 앱** 그 사람이 탄 구간만큼의 금액·거리·전기량을 확정한다. 이후 구간은 남은 사람끼리 나눈다 정차 판단(차속 · 기어) · 구간별 실측
- **얻는 것** **"먼저 내린 사람은 얼마?"가 그 자리에서 끝난다**



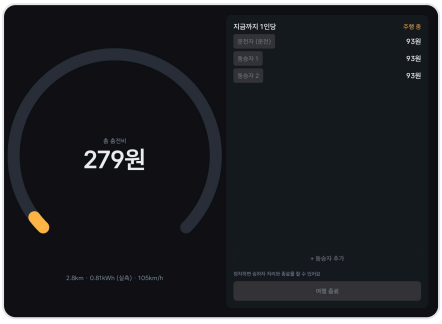
4 도착 — 정산서와 공유 정차 중

- **기존 불편** 며칠 뒤 운전자가 계산해 청구하거나, 그냥 포기한다
- **사용자** "여행 종료"를 누른다. 동승자는 QR을 찍어 내 몫을 고르고 금액을 복사해 쓰던 송금 앱에 붙여 넣는다
- **차 · 앱** 총액·실제 전비, 사람별 금액, 구간별 내역(실측/추정), 지나온 길과 하차 지점을 보여 준다 위치 경로 · NaviHelper 지명 · 웹 정산서
- **얻는 것** **계산과 청구가 사라진다**

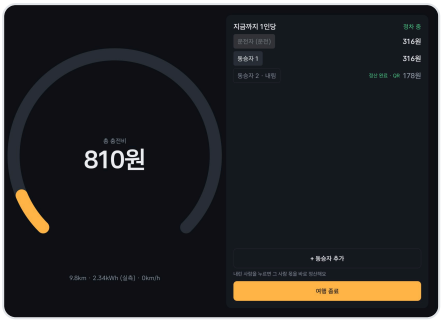


화면은 모두 Pleos Connect 에뮬레이터에서 실제 SDK 값으로 찍었다. 중간에 사람을 태울 때도 같다 — 정차 중 "+ 동승자 추가"로 새 구간을 시작하고, 새로 탄 사람은 그때부터 함께 탄 구간만 낸다(→ 07). 시연 영상은 이 네 단계를 그대로 따른다.

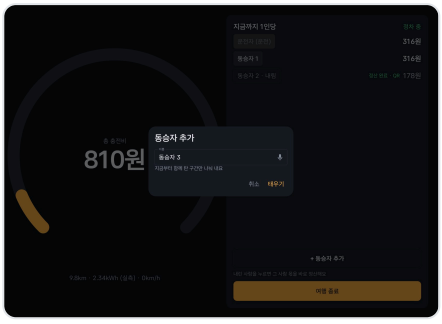
달리는 동안은 큰 숫자 하나만 보이고, 서 있을 때만 조작이 열린다.



주행 중 금액만 오른다. 이름 버튼·동승자 추가·여행 종료가 흐리게 잠긴다.



정차 중 같은 화면에서 조작이 열린다. 내린 사람은 "정산 완료"로 남는다.



정차 중 동승자 추가. 이름은 마이크로 말한다. "지금부터 함께 탄 구간만 나눠 내요."

기능	주행 중	정차 중	이렇게 나눈 이유
실시간 금액·거리·소모량 보기	●	●	보기만 하고 조작이 없다. 큰 숫자 하나로 한눈에 읽힌다
기록 (백그라운드)	●	●	포그라운드 서비스가 자동으로 기록한다. 화면을 볼 필요가 없다
하차 정산 · QR	—	●	사람이 실제로 내리는 순간은 정차 중이다
동승자 추가 (음성 이름 입력)	—	●	입력이 필요한 조작이다
여행 종료 · 결과 확인	—	●	결과 화면은 정보량이 많다

핵심 기능 세 가지

① 실측 구간 기록

1초마다 차량 값을 읽어 구간에 쌓는다. **시동을 껐다 켜면** 부팅과 함께 진행 중이던 여행을 이어 기록하고, 꺼져 있던 동안의 거리는 주행거리계로 보정한다.

② 하차·탑승 즉시 정산

정차 중 이름을 누르면 그 사람의 몫이 **확정**되고, 이후 구간은 남은 사람끼리 나눈다. 중간에 탄 사람은 그때부터만 낸다(최대 5명). 운전자 몫은 켜고 끌 수 있다.

③ 설치 없는 QR 정산서

차 화면의 QR을 폰 카메라로 찍으면 브라우저에서 정산서가 열린다. 내 몫을 골라 **금액을 복사**한다. 정산 내용은 주소의 # 뒤에만 담기고 서버에 저장하지 않는다.

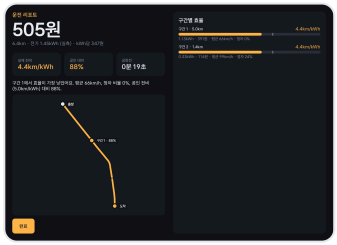
솔로 모드 혼자 탈 때도 여는 이유 (보조 기능)

같은 실측 데이터로 **구간별 실제 전비·연비, 비용, 평균 속도, 정차 비율**을 보여 주고, 공인 효율 대비 점수와 효율이 가장 낮았던 구간을 알려 준다.

구간은 차가 스스로 나눈다 — **2분 이상 정차**, **시내(50km/h 미만)↔고속(80km/h 이상) 전환이 2분 이상** 이어질 때, 지역(시·군·구)이 바뀔 때.



주행 중 — 고속 구간으로 바뀌자 구간 2가 시작됨



운전 리포트 — 실제 전비 4.4km/kWh, 공인 대비 88%

주의 분산을 줄이는 설계 원칙

운전 중 입력 0회 (조작은 정차 중에만) · **큰 숫자 하나** (주행 화면의 정보는 총액과 1인당 금액뿐) · **손 대신 음성** (이름은 SpeechToText로) · **알아서 기록** (앱을 보지 않아도 서비스가 기록). 주행 중에도 금액을 띄우기 위해 Distraction Optimization을 선언했고, 그만큼 주행 화면에는 입력·QR·긴 글을 두지 않는다. 구현

차량 신호가 1초마다 샘플이 되고, 샘플이 구간이 되고, 구간이 금액이 되어 차 화면과 QR로 나간다.



실선은 구현되어 동작하는 흐름, 점선은 개발 예정이거나 뒤에서 받치는 부분이다. 모든 차량 값은 실제 Pleos SDK에서 온다 — 앱 코드에는 가짜(시뮬레이션) 데이터 구현이 없고, 자동 검사가 이를 막는다.

값의 신뢰도를 지키는 규칙

- **출처 우선순위** — 차량 잔량 변화(실측) → 값이 없을 때만 공인 효율(추정). 추정 구간은 화면에 "추정"으로 표시한다.
- **연료 게이지 해상도** — 연료 잔량은 짧은 구간에서 오차가 커서 20km 이상 구간에서만 쓴다. 배터리는 Wh 단위로 촘촘해 제한이 없다.
- **거리** — 주행거리계를 쓰고, 값이 움직이지 않으면 차속을 적분한다. 시동이 꺼져 있던 동안의 거리도 주행거리계로 보정한다.
- **시간** — 샘플 간격은 시계 변경에 영향받지 않는 단조 시계로 잰다.
- **네트워크 없이도 정산** — 계산은 모두 차 안에서 한다. 가격은 마지막으로 받은 값을 날짜와 함께 쓴다.

모듈 구조와 권한

```
:app ──▶ :pleos ──▶ :domain
  │  └─▶ :data  ──▶ :domain
  └──────────────▶ :domain  (순수 Kotlin)
```

- 정산·구간 규칙은 Android와 분리된 순수 Kotlin 모듈이라 차 없이 단위 테스트로 검증한다.
- SDK 호출·파일·네트워크는 IO 스레드에서 한다.
- 권한 — Pleos `CAR_DRIVE` `CAR_ENERGY` `NAVI_ROUTE` `STT_SERVICE`, AAOS `CAR_SPEED` `CAR_INFO`, 위치, 마이크

시동을 껐다 켜도 이어지는 기록 휴게소·충전소에 들러도 여행은 하나

주행 중
5초마다 진행 상태를 저장한다.
쓰다 꺼져도 깨지지 않게 통째로
바꿔 쓴다

→

시동 OFF
앱이 끝나도
마지막 상태가 남는다

→

시동 ON
부팅 신호를 받아, 앱을 열지
않아도
여행을 복원한다

→

거리 보정
꺼져 있던 동안 주행거리계가
늘었으면 그만큼 더한다

데이터가 머무는 곳

데이터	저장 위치	밖으로 나가는가
여행 기록 · 지나온 길	차량 기기 안 (Room · 파일)	나가지 않는다
이름 · 사람별 금액	QR 주소의 # 뒤	동승자 폰 브라우저까지만 간다. # 뒤 내용은 서버로 전송되지 않는다
유가 · 충전 단가	가격 API 응답	차량·개인 정보 없이 가격만 내려받는다

아이디어가 아니라 이미 도는 앱이다. 기능마다 "어떤 환경에서 무엇을 확인했는지"를 적는다.

검증 환경 ① 에뮬레이터

Pleos Connect 에뮬레이터 v2.1 (아이오닉 5, API 34). 기어·속도는 Virtual Console로 조작한다. 앱은 **실차와 같은 Pleos SDK**로만 값을 읽는다.

검증 환경 ② 주행 재현

실제 도로 경로를 따라 위치·주행거리·배터리를 함께 움직이는 재현 도구. 배터리 감소는 속도에 맞춘 물리 모델(공기저항·가속·회생제동·공조)을 따른다.

검증 환경 ③ 자동 테스트

단위 테스트 46개 전부 통과 (2026-11-10 실행) — 정산 15 · 구간 기록 20 · 여행 진행 4 · 차종 판별 3 · 운전 리포트 2 · 공유 링크 2

기능	사용 기술 (SDK · API)	상태	확인한 환경 → 결과
전기 소모량 실측	Vehicle `EvBattery.registerEvBatteryLevel`	구현 완료	에뮬레이터 주행 → 9.8km에서 2.34kWh 실측
주행거리	Vehicle `Odometer.registerDistanceDriven`	구현 완료	에뮬레이터 주행 + 단위 테스트
정차 판단 · 조작 잠금	Vehicle `Gear` + AAOS `PERF_VEHICLE_SPEED`	구현 완료	주행 중 잠김, 정차 시 열림 확인 (07 화면)
구간 정산 · 하차 확정 · 중간 탑승	순수 Kotlin 도메인 계산 (1원 단위)	구현 완료	단위 테스트 35개 + 에뮬레이터 시연 (05)
시동 OFF/ON 이어하기	원자적 저장 + 부팅 리시버 + 주행거리계 보정	구현 완료	에뮬레이터 재부팅 뒤 여행·하차 금액 유지
이름 음성 입력	SpeechToText (ON_DEVICE)	구현 완료	에뮬레이터 마이크로 이름 입력 확인
오늘 유가	서버리스 함수 → 오피넷 전국 평균 (키는 서버에만)	구현 완료	11/10 휘발유 1,855원 · 경유 1,842원/L 수신
QR 정산서	QR → 정적 웹 페이지 (주소 해시), 금액 복사	구현 완료	배포된 정산서에서 내 몫 선택·복사 확인
솔로 자동 구간 · 리포트	정차 2분 / 도로 유형 전환 2분 / 지역 변경	구현 완료	단위 테스트 + 에뮬레이터 주행 (07 화면)
차종 판별	Vehicle 배터리·연료 값 유무 (경유는 사용자 선택)	구현 완료	전기차 판별 확인, 하이브리드는 단위 테스트
연료 소모량 실측	Vehicle `Fuel.registerFuelPercent` × 탱크 용량	프로토타입	에뮬레이터가 전기차라 단위 테스트로 검증
지명 · 지역 구간 분할	NaviHelper `getCurrentLocationInfo` + 위치	프로토타입	주소 수신 확인. 지역 변경 분할은 단위 테스트
충전 단가	환경부 공용 급속 고시값 (출발 화면에서 수정)	프로토타입	347.2원/kWh 적용. 집·완속은 직접 수정
하차 금액 음성 안내	TextToSpeech SDK	개발 예정	2차 PT 전 추가
순간 연비 기반 실측	Vehicle 순간 연비	개발 예정	단위 확인 뒤 적용 여부 결정

구현 완료 코드와 테스트 결과가 있는 기능 **프로토타입** 시연은 되지만 환경·데이터가 제한된 기능 **개발 예정** 기술 검증·개발이 남은 기능

아직 확인하지 못한 것

공모전의 검증 환경은 Pleos Connect 에뮬레이터이고, 에뮬레이터 차량은 전기차 한 종이다. 그래서 휘발유·하이브리드의 연료 실측과 지역 변경 분할은 단위 테스트로만 검증했다. 실차 확인은 App Market 탑재 준비 단계의 첫 과제다. → 12

직접 확인할 수 있는 증거

① **시연 영상** — 06의 네 단계를 그대로 ② **GitHub 저장소** — 소스, APK, 실행 방법, 테스트 ③ **본 문서의 화면** — 모두 에뮬레이터 실측

재는 것, 나누는 것, 구간으로 끊는 것은 각각 이미 있다. 이 셋을 차 안에서 하나로 이은 서비스는 찾지 못했다.

2026-11-10 기준 공개 자료로 확인한 범위다. "최초"나 "유일"을 주장하지 않고, 겹치는 부분을 먼저 인정한다.

유형	대표 서비스	금액은 어디서 오나	구간 분할	하차 때 확정	우리와 겹치는 점 → 다른 점
모바일 정산	토스 더치페이, 카카오페이 정산하기, 카카오뱅크 1/N 빵나누기, Splitwise	사람이 총액을 입력	—	—	나누기·송금 요청, 링크 공유 → **나눌 금액을 만들어 주지 않는다.** 우리는 금액만 만들고 송금은 이 앱들에 맡긴다 (보완 관계)
유류비 계산	지도 길찾기의 예상 유류비, 웹 유류비 계산기	평균 유가 × 가정한 연비 (추정)	—	—	거리 기반 연료비 → 출발 전 추정치라 실제 소모와 중간 하차를 반영하지 못한다
구간별 여행 정산	RapidSplit 등 구간(leg) 분할 앱·계산기	구간별 거리·연료비를 사람이 입력	●	—	"그 구간에 탄 사람만 분담"하는 논리 → 차량 데이터가 없어 입력이 필요하고, 여행이 끝난 뒤 정산한다
호출 · 카풀 플랫폼	타다 N분의 1 정산, 반반택시, BlaBlaCar	플랫폼 운임 · 거리 요율	●	●	"이동한 만큼 낸다"는 원칙 → 호출·예약 서비스의 운임이지, 내 차가 실제로 쓴 연료·전기가 아니다
차량 기본 기능	트립 컴퓨터, 제조사 앱의 운행 정보·차계부	차량 실측 (거리·연비까지, 금액 없음)	—	—	실측 데이터라는 재료 → 금액으로 바꾸지 않고, 사람별로 나누지 않는다
실차 주행 비용	Tessie·TeslaFi, OBD 주행기록 앱	차량 실측 × 단가 (운전자용 기록)	—	—	실제 소모로 주행 비용을 계산 → 동승자 분배가 없고, 폰·클라우드에서 동작한다
AUTONOMICS	Pleos Connect 차량 앱	**차량 실측(구간별 소모량) × 오늘 단가**	●	●	재기 + 구간 분할 + 하차 확정을 **차 안에서 한 번에**. 동승자는 설치 없이 QR

어디에 서 있는가



예상 질문과 답

"토스·카카오페이로 하면 되지 않나?"	그 앱들은 "나눌 금액 입력"에서 시작한다. 우리는 그 금액을 만든다. 송금은 기존 앱이 더 잘하므로 금액 복사로 넘긴다.
"차에 트립 컴퓨터가 있지 않나?"	거리·연비까지만 보여 준다. 금액으로 바꾸고 사람별로 나누는 단계가 없다.
"탄 만큼 내는 건 이미 있지 않나?"	호출 서비스의 운임 정산에는 있다. 내 차로 가는 지인 동승에서 "실제 소모"를 나누는 것은 다른 문제다.
"구간별로 나누는 앱도 있던데?"	있다. 다만 거리와 연료비를 사람이 입력해야 한다. 우리는 그 입력을 차량 데이터로 없앴다.

같은 차량 데이터를 쓰더라도 달라지는 지점

① **평균이 아니라 차이** — "평균 전비 × 거리"로 환산하면 그것도 추정이다. 우리는 구간이 시작되고 끝날 때의 실제 잔량 차이를 쓴다. ② **나중이 아니라 하차 순간** — 내린 사람의 금액은 그 자리에서 확정되고, 이후 구간이 영향을 주지 않는다. ③ **정기 모임이 아니라 그때그때** — 가입·매칭 없이, 오늘 같이 탄 사람끼리 바로 쓴다. ④ **값의 출처 표시** — 실측과 추정을 화면에서 구분한다.

사실 확인한 자료 — 토스 공식 안내(더치페이), 카카오뱅크 1/N 빵나누기 보도(2026-01), 네이버 Directions 5 API 문서(유류비 산정 기준), 현대·기아 오너스 매뉴얼(트립 컴퓨터), 마이현대 안내, 타다 정산 기능 보도(2023-03), 서울시 반반택시 자료(2019), BlaBlaCar 약관, Tessie 도움말, RapidSplit 소개. 일부 서비스의 현재 운영 여부는 확인하지 못했다.

동승자에게 건네는 QR이 다음 사용자를 데려오고, 솔로 모드가 다음 운전에 다시 열게 한다.

아직 실제 사용자가 없다. 그래서 매출 전망 대신 누가, 얼마나 자주, 왜 다시 쓰는지와 그것을 어떻게 잴지를 적는다.

항목	내용
타깃 고객 분석	Pleos Connect 탑재 차량 운전자 중 **동승 이동이 잦은 사람**. ① 주말 여행·레저를 지인과 가는 운전자 (금액이 크다) ② 출퇴근 카풀 운전자 (반복된다) ③ 전기차 운전자 — 충전 영수증이 이동과 맞지 않아 문제가 가장 크므로 첫 공략 대상이다
사용자 확보	① App Market에서 "기름값·충전비 나누기"라는 분명한 쓸모로 발견된다 ② **동승자 QR** — 정산서를 받은 동승자가 앱을 알게 되고, 자기 차가 Pleos면 설치한다. 동승자는 설치 없이 결과를 받으므로 첫 경험에 장벽이 없다
이용 빈도	정산 모드는 동승 이동이 있을 때마다 쓴다 (여행은 월 단위, 카풀은 주 단위 가정). 솔로 모드는 혼자 타는 매 운전에 쓴다
확대 계획	**1단계** 지인 동승의 연료·충전비 → **2단계** 통행료·주차비 등 이동 비용 전체 → **3단계** 카셰어링·렌터카·법인차의 주행 비용 정산과 리포트
수익 모델	개인 사용자는 무료로 두고 사용자 수와 재사용을 먼저 검증한다. 이후 법인 주행비 리포트(구독)와 카셰어링 연동(B2B)을 검토한다

성장 루프 쓸수록 다음 사용자가 생기는 구조



혼자 탈 때 — 솔로 모드
매 운전의 실제 전비·연비 리포트가 동승이 없는 날에도 앱을 열게 한다

예상 MAU와 Retention Rate 가정

예상 MAU = 도달 가능 고객 × 설치 전환율 × 월간 활성화율

Pleos Connect 차량 10만 대당	설치 전환율	월간 활성화율	MAU	30일 재사용률
보수	3%	40%	1,200	25%
기본	5%	50%	2,500	35%
낙관	8%	60%	4,800	45%

모든 비율은 검증 전 가정이며 출시 후 실제 지표로 바꾼다. 30일 재사용률(Retention Rate)은 처음 쓴 날로부터 30일 안에 다시 쓴 운전자의 비율이다. 정산 모드만으로는 동승 이동이 있는 달에만 돌아오므로, 그 사이를 솔로 모드가 메운다는 전제다.

다시 쓰게 만드는 것 (리텐션)

① 혼자 탈 때의 운전 효율 리포트 ② 지난 정산과 사람별 정산서 다시 보기 ③ 같은 멤버가 반복되는 카풀의 습관화 ④ 한 번 정확한 금액을 받아 본 동승자의 "이번에도 그걸로 하자"

출시 후 재는 지표

정산 완료 여행 수 · 여행당 동승자 수 · **QR 스캔률**(동승자 도달) · 솔로 모드 주간 사용 일수 · **30일 재사용률**. 2차 PT 전에는 지인 테스트로 QR 스캔률과 재사용 의향부터 확인한다.

1차 제출 시점에 핵심 기능은 이미 동작한다. 남은 일은 실제 사용자에게 확인하고 시연을 다듬는 것이다.

개발 로드맵 MVP → 검증 → 고도화

① MVP — 완료

10/31 착수 ~ 11/15 1차 제출

- 정산 모드: 구간별 실측 정산, 하차 확정, 중간 탑승, QR 정산서
- 솔로 모드, 시동 OFF/ON 이어하기
- 음성 이름 입력, 오늘 유가
- 시연 영상 · GitHub(APK) 제출

② 검증

11/16 ~ 12/3 2차 PT

- 지인 동승 테스트와 인터뷰 반영 (→ 03)
- TextToSpeech 하차 금액 안내
- 충전 단가 최신화, 데려다주기 정책 결정
- 2차 PT 시연 시나리오 고정 (에뮬레이터)

③ 고도화

2차 PT 이후

- 순간 연비 기반 실측 (연료차 정밀도)
- 통행료·주차비 등 이동 비용 전체로 확대
- 법인 주행비 리포트, 카셰어링 연동
- 실차 검증 (App Market 탑재 준비)

주요 리스크와 대응

리스크	대응
차종별 데이터 차이 (연료 게이지 해상도)	짧은 구간은 공인 연비로 추정하고 "추정"으로 표시한다. 순간 연비 활용을 검토한다
휘발유·하이브리드 실측 미검증	검증 환경인 에뮬레이터가 전기차 한 종이라 단위 테스트로 검증했다. 실차 확인은 App Market 탑재 준비 단계에서 가장 먼저 한다
충전 단가가 실제와 다름 (집·완속·급속)	출발 화면에서 단가를 직접 고칠 수 있다. 기본값은 최신 고시로 갱신한다
네트워크 단절	정산은 오프라인으로 계산하고, 가격은 마지막 값을 날짜와 함께 쓴다. QR 정산서는 서버가 필요 없다
개인정보	이름·금액은 QR 주소의 # 뒤에만 있고 서버에 저장하지 않는다. 경로는 기기 안에만 저장한다
운전자 주의 분산	주행 중 조작 잠금, 음성 입력, 승하차 처리는 정차 중에만
자가용 유상운송 규제와의 관계 여객자동차 운수사업법 제81조	모르는 사람을 연결하거나 요금을 받는 서비스가 아니라, 함께 이동한 지인끼리 **실제 든 연료·전기 값만** 나누는 계산 도구다. 이익을 붙이지 않고, 운전자도 분담에 포함하는 것이 기본값이며, 알선·결제 기능이 없다. 다만 법이 운행 경비도 "유상"에 포함하므로 출시 전 법률 검토로 허용 범위를 확인한다
데려다주기(우회 운전) 비용	현재는 실제로 탄 구간만 나눈다. 내려 준 뒤 돌아오는 거리를 어떻게 나눌지는 사용자 인터뷰 뒤 정책으로 정한다

팀과 실행 역량

역할 구성만 기입 (예: 기획·개발 1명). 이름·소속·연락처는 쓰지 않는다

사실 착수 12일 만에 핵심 기능을 구현했다 (10/31 첫 커밋 → 11/10 검증). Kotlin 4,000여 줄, 4개 모듈, 단위 테스트 46개. 모듈 의존 방향과 "가짜 데이터 금지"는 커밋 때마다 자동 검사한다.

심사 기준별 근거 위치

심사 항목	공식 평가 내용	근거 쪽
차량특화성	모바일 대비 차량만의 독자적 가치와 우위 · 주행·정차 맥락 및 차량 기능과의 결합	04 · 06 · 07
창의성	독창적인 아이디어 · 운전자·동승자의 Pain Points & Needs 정의와 해결 방식	02 · 03 · 05 · 10
기술 활용성	Pleos SDK·API를 핵심 기능과 사용 경험에 적절히 활용	08 · 09
사업성	타깃 고객, 사용자 확보, 지속 이용·확대 (MAU · Retention Rate · 확대 계획)	11
가산점	개발 산출물 — 시연 영상, GitHub 저장소(소스·APK)	09

본 문서의 화면은 모두 Pleos Connect 에뮬레이터(v2.1, 아이오닉 5)에서 실제 Pleos SDK로 받은 값으로 찍었다. 시장 규모·전환율 등 검증 전 수치는 "가정"으로 표시했고, 출처 없는 수치를 사실로 쓰지 않았다.